Vergleich der Robustheit und Anpassungsfähigkeit von Deep Q-Networks und REINFORCE im Falle von Triebwerksausfällen in der LunarLander-Umgebung

Niklas Schneider

HAW Hamburg - Department Informatik
Berliner Tor 5, 20099 Hamburg
niklas.schneider@haw-hamburg.de

Zusammenfassung. Diese Arbeit untersucht die Robustheit und Anpassungsfähigkeit der Reinforcement-Learning-Verfahren Deep Q-Network (DQN) und REINFORCE bei simulierten Triebwerksausfällen in der Lunar Lander-Umgebung. Während DQN in stabilen, deterministischen Umgebungen schneller lernt und höhere Belohnungen erzielt, zeigt es bei unerwarteten Störungen Schwächen. REINFORCE ist jedoch robuster gegenüber dynamischen Veränderungen, insbesondere nach zusätzlichem Training. Ohne Nachtraining verschlechtert sich die Leistung beider Methoden, wobei DQN deutlich stärker betroffen ist. Durch gezieltes Training kann REINFORCE seine Leistung signifikant verbessern und in einigen Szenarien sogar das ursprüngliche Leistungsniveau übertreffen, während DQN kaum Anpassungen an die veränderten Umweltbedingungen vornimmt. Die Ergebnisse zeigen, dass policybasierte Verfahren wie REINFORCE für unsichere und dynamische Umgebungen besser geeignet sind, während wertebasierte Methoden wie DQN in stabilen, vorhersehbaren Szenarien ihre Stärken ausspielen.

1 Einführung

Die Anwendung von Reinforcement Learning (RL) zur Lösung komplexer Steuerungsprobleme wie beispielsweise der Navigation und Landung einer Mondfähre in der LunarLander-Umgebung ist ein intensiv erforschtes Gebiet [1, 2, 3]. Verschiedene RL-Methoden haben sich als vielversprechend erwiesen, um intelligente Agenten zu trainieren, die in der Lage sind, optimale Entscheidungen in dynamischen Umgebungen zu treffen [4, 5]. In vielen realen Anwendungen, insbesondere in physikalischen Systemen, können unerwartete Störungen und Unsicherheiten auftreten, welche die Leistung und Zuverlässigkeit trainierter Agenten erheblich beeinträchtigen können [6, 7]. Die vorliegende Arbeit untersucht die Robustheit und Anpassungsfähigkeit zwei etablierter RL-Verfahren: die wertebasierte Methode Deep Q-Network (DQN) und die policybasierten Methode REINFORCE, im Falle von simulierten Triebwerksausfällen in der LunarLander-Umgebung.

2 Problemstellung

Das Ziel dieser Arbeit ist es, die LunarLander-Umgebung mithilfe des wertebasierten Verfahrens DQN sowie mit dem policybasierten Verfahren REINFORCE zu lösen und anschließend Unsicherheiten in Form von simulierten Triebwerksausfällen einzuführen. Anhand dieser Simulationen soll untersucht werden, welches der beiden Verfahren besser in der Lage ist, sich an unerwartet auftretende Systembedingungen anzupassen. Im Mittelpunkt der Ausarbeitung stehen insbesondere folgende Forschungsfragen:

1. Wie unterscheiden sich die beiden Verfahren hinsichtlich ihrer Fähigkeit, die LunarLander-Umgebung ohne Unsicherheiten effektiv zu lernen?
2. Wie robust zeigen sich die Verfahren gegenüber simulierten Triebwerksausfällen?
3. Welches Verfahren weist bei simulierten Triebwerksausfällen eine schnellere Anpassungsfähigkeit auf?

Um diese Fragen zu beantworten, werden zunächst Vanilla-Agenten für beide Verfahren in der ursprünglichen LunarLander-Umgebung trainiert. Die Anpassungsfähigkeit dieser Agenten wird anschließend in einer modifizierten Umgebung mit simulierten Triebwerksausfällen evaluiert, sowohl ohne vorheriges Training in der gestörten Umgebung als auch nach gezieltem Training.

3 Verwandte Arbeiten

Die Anwendung von RL zur Lösung des LunarLander-Problems ist ein intensiv erforschtes Gebiet. Verschiedene RL-Techniken wurden eingesetzt, um intelligente Systeme zu trainieren, die in der Lage sind, eine Mondlandefähre zu steuern und sicher auf der Mondoberfläche zu landen. Eine Studie [4] vergleicht DQN, Double DQN (DDQN) und Policy Gradient und stellt fest, dass DQN und DDQN in Bezug auf Belohnungen und Stabilität überlegen sind, während Policy Gradient schnellere Ergebnisse liefert. Eine andere Arbeit [1] befasst sich mit Q-Learning, DQN und DDQN und zeigt, dass Q-Learning effizienter ist, während DDQN bessere Stabilität und Belohnungen liefert. Die Studie von [7] untersucht DQN und Sarsa in Szenarien mit Unsicherheiten und zeigt, dass DQN robuster bei verrauschten Beobachtungen arbeitet. Es gibt auch Untersuchungen [6], die zeigen, dass ein neues Belohnungssystem die Effizienz von DQN steigert und zu besseren Ergebnissen in Bezug auf Treibstoffverbrauch und Erfolgsquote führt. Der Einsatz von DQN mit Kurzzeitgedächtnis ermöglicht eine genauere Schätzung des Umweltzustands bei komplexen Unsicherheiten [3]. Schließlich gibt es Studien [2], die DQN und Policy Gradient in deterministischen Umgebungen untersuchen und feststellen, dass DQN überlegen ist, wobei in diesen Studien auch angemerkt wird, dass wertebasierte Methoden unter Unsicherheiten in realen Szenarien möglicherweise bessere Ergebnisse erzielen könnten.

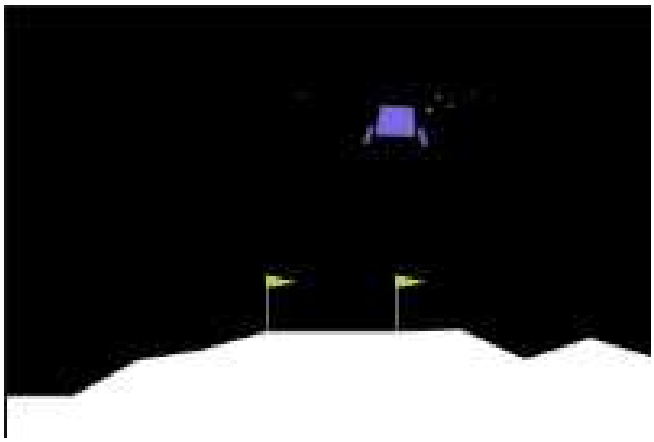


Abbildung 1: LunarLander-Umgebung

4 Grundlagen

4.1 Umgebung

4.1.1 LunarLander

Das Framework »Gymnasium« von Farama für die Simulation des LunarLander-Problems ist ein Fork des OpenAI Gym, das verschiedene Umgebungen, z. B. Atari-Videospiele oder Robotik, zur Verfügung stellt. Farama-Foundation ist eine gemeinnützige Organisation, die sich der Förderung von RL durch bessere Standardisierung und Open-Source-Tools widmet. Sie unterhalten drei weit verbreitete Kernbibliotheken, die standardisierte APIs für verschiedene RL-Umgebungen bereitstellen. Darunter sind Robotik-Simulationen und Videospiele [8]. Die Lunar Lander-Umgebung ist eine Simulation einer Landefähre auf dem Mond. Dabei simuliert die Umgebung Herausforderungen, eine Landefähre sicher auf der Mondoberfläche zu landen, und berücksichtigt dabei physikalische Aspekte wie Schwerkraft und Trägheit. Ein Agent steuert eine Mondlandefähre, mit dem Ziel, sicher auf einer definierten Landeplattform zu landen (siehe Abbildung 1). Die Landeplattform befindet sich immer bei den Koordinaten (0,0), wobei es möglich ist, dass der Agent die Mondlandefähre außerhalb der Landeplattform landet bzw. diese zum Absturz bringen kann [9].

4.1.2 Aktionsraum

Es existieren zwei Varianten der LunarLander-Umgebung, eine mit diskretem und eine mit kontinuierlichem Aktionsraum. In diesem Projekt wird der diskrete Aktionsraum verwendet. Die Triebwerke können an oder ausgeschaltet werden und zünden mit vollem Schub. Die Mondlandefähre verfügt über drei Triebwerke, zwei an den Seiten rechts und links, sowie ein Haupttriebwerk unter der Mondlandefähre. Der Agent verfügt über vier diskrete Aktionen [9]:

0: Nichts tun (keine der drei Triebwerke werden gezündet)

1: Linkes Steuerungstriebwerk zünden

2: Haupttriebwerk zünden

3: Rechtes Steuerungstriebwerk zünden

4.1.3 Zustandsraum

Die Mondlandefähre ist durch einen 8-dimensionalen Vektor beschrieben, der verschiedene Attribute enthält [9]

$$\text{Zustandsvektor} \rightarrow \begin{cases} x: & \text{Koordinate in } x \\ y: & \text{Koordinate in } y \\ v_x: & \text{Lineare Geschwindigkeit von } x \\ v_y: & \text{Lineare Geschwindigkeit von } y \\ \theta: & \text{Winkelorientierung} \\ v_\theta: & \text{Winkelgeschwindigkeit} \\ Boolean: & \text{Bodenkontakt linkes Landebein} \\ Boolean: & \text{Bodenkontakt rechtes Landebein} \end{cases}$$

4.1.4 Belohnungssystem

Das Belohnungssystem ist so gestaltet, dass es die Agenten für eine sichere und effektive Landung belohnt:

- erhöht/verringert sich mit der Entfernung der Landefähre zur Landeplattform
- erhöht/verringert mit der Geschwindigkeit der Landefähre
- verringert mit der Schieflage der Landefähre
- erhöht um 10 Punkte für jedes Landebein mit Kontakt zur Mondoberfläche
- verringert um 0.03 Punkte für jedes Zünden pro Frame der seitlichen Triebwerke
- verringert um 0.3 Punkte für jedes Zünden pro Frame des Haupttriebwerks

Jede Episode erhält eine zusätzliche Belohnung von -100 oder +100 für das sichere Landen. Eine Episode gilt als erfolgreich, wenn eine Gesamtbelohnung von mindestens 200 Punkten erreicht wird [9].

4.1.5 Anfangsbedingung und Endbedingung

Die Mondlandefähre startet im oberen mittleren Bereich des Sichtfeldes mit einer zufälligen Kraft, die auf den Schwerpunkt wirkt. Eine Episode endet, wenn einer der folgenden Zustände eintritt:

- Die Mondlandefähre stürzt ab
- Die Mondlandefähre verlässt das Sichtfeld
- Die Mondlandefähre bewegt sich nicht mehr

Diese Bedingungen gewährleisten, dass der Agent lernt, innerhalb der definierten Grenzen zu operieren und eine sichere Landung durchzuführen [9].

4.2 Wertebasierte Methode: DQN

4.2.1 Q-Learning

Q-Learning ist ein modellfreier Algorithmus, bei dem es darum geht, eine optimale Policy zu finden, die dem Agenten hilft zu entscheiden, welche Aktionen in welchem Zustand ausführen soll [10, 7]. Das Ziel des Agenten ist es, die kumulative zukünftige Belohnung zu maximieren [11]. Q-Learning basiert auf dem Konzept der Q-Funktionen, das die erwartete kumulative Belohnung für die Ausführung einer bestimmten Aktion in einem bestimmten Zustand und das anschließende Befolgen einer optimalen Policy darstellt [11]. Die optimale Q-Funktion wird nach der Bellman-Gleichung iterativ aktualisiert:

$$Q(s,a) = \mathbb{E}\left[r + \gamma \max_{a'} Q(s',a') \mid s,a\right] \tag{1}$$

wobei s der aktuelle Zustand ist, a die ausgeführte Aktion, r die erhaltene Belohnung, γ der Diskontierungsfaktor, s' der nachfolge Zustand und a' die nächste Aktion ist [10, 11]. Die Q-Funktion wird iterativ durch folgende Regel geschätzt:

$$Q(s,a) \leftarrow Q(s,a) + \alpha\left[r + \gamma \max_{a'} Q(s',a') - Q(s,a)\right] \tag{2}$$

wobei α die Lernrate repräsentiert (Mnih et al., 2013). Da Q-Learning ein sogenanntes »off-policy« Verfahren ist, lernt es die optimale »greedy policy« [7]. Um ein angemessenes Maß an Exploration des Zustandsraums zu gewährleisten, wird häufig eine ϵ-greedy-Policy eingesetzt, bei der zufällige Aktionen (Exploration) mit einer Wahrscheinlichkeit von ϵ gewählt wird und mit einer Wahrscheinlichkeit von 1-ϵ die beste Aktion (Exploitation) gewählt wird [10, 11, 7].

4.2.2 DQN

DQN erweitert Q-Learning, um ein Deep-Neural-Network, um die Beschränkung des Q-Learning für kontinuierliche Zustandsräume zu beheben, um die Q-Werte zu approximieren [6]. Das Netzwerk nimmt den aktuellen Zustand (in der Lunar Lander-Umgebung 8 Dimensionen) als Eingabe und gibt die Q-Werte für

alle Zustand-Aktions-Paare aus [7, 6]. Der optimale Q-Wert $Q^*(s, a)$ wird mithilfe des neuronalen Netzes mit Parameter θ geschätzt. Die Verlustfunktion zur Optimierung des Netzwerks basiert auf dem mittleren quadratischen Fehler des Temporal-Differences-Error:

$$L_i(\theta_i) = \mathbb{E}_{(s,a)\sim\rho(\cdot)}\left[(y_i - Q(s, a; \theta_i))^2\right], \tag{3}$$

wobei der optimale Zielwert durch folgende Funktion repräsentiert wird:

$$y_i = \mathbb{E}_{s'\sim\mathcal{E}}\left[r + \gamma \max_{a'} Q(s', a'; \theta_{i-1})\right]. \tag{4}$$

Dabei bezeichnet θ_{i-1} die Netzwerkparameter der vorherigen Episode, $\rho(\cdot)$ eine Wahrscheinlichkeitsverteilung über Zustands-Aktions-Paare und ϵ die Verteilung der Folgezustände [7, 6].

4.3 Policy-Gradient-Methode: REINFORCE

REINFORCE ist eine modellfreie Policy-Gradient-Methode, die darauf abzielt, eine optimale Policy zu finden, indem sie die erwartete kumulative Belohnung direkt maximiert [12]. Das Ziel dieser Methode ist die direkte Maximierung der erwarteten kumulativen Belohnung, indem Policy-Parameter in Richtung höherer Belohnungen angepasst werden [13]. Die Zielfunktion $J(\theta)$ ist definiert als:

$$J(\theta) = \mathbb{E}_{\tau\sim\pi_\theta}\left[\sum_{t=0}^{T} \gamma^t R(s_t, a_t)\right], \tag{5}$$

wobei θ die Parameter der Policy (z. B. die Gewichte eines neuronalen Netzes) darstellt, τ eine Trajektorie, die der Policy π_0 folgt und $R(s_t, a_t)$ die Belohnung zum Zeitpunkt t im Zustand s_t bei der Ausführung der Aktion a_t [13]. Das Policy Gradient Theorem ist die Grundlage für REINFORCE und gibt den Gradienten der erwarteten Gesamtbelohnung $J(\theta)$ bezüglich der Policy-Parameter θ an:

$$\nabla_\theta J(\theta) = \mathbb{E}_{\tau\sim\pi_\theta}\left[\sum_{t=0}^{T} \nabla_\theta \log \pi_\theta(a_t|s_t) G_t\right] \tag{6}$$

Hierbei ist G_t die akkumulierte Belohnung ab dem Zeitpunkt t [12]. Die daraus resultierende Update-Regel für die Policy-Parameter lautet:

$$\theta \leftarrow \theta + \alpha \sum_{t=0}^{T} G_t \cdot \nabla_\theta \log \pi_\theta(a_t|s_t), \tag{7}$$

Für die Verwendung von REINFORCE mit Deep-Neural-Networks kann die Policy-Optimierung durch die Multi-Class Cross-Entropy Loss-Funktion erfolgen. Dazu muss die Label-Verteilung eine Wahrscheinlichkeitsverteilung sein,

weshalb eine modifizierte Zielverteilung Y_{st} erstellt wird [13]. Diese entsteht, indem die ursprüngliche Wahrscheinlichkeitsverteilung $\pi_\theta(\cdot \mid s_t)$ durch den Return G_t in Richtung der tatsächlich gewählten Aktion $\bar{a}$ angepasst wird:

$$Y_{s_t}(\cdot) = (1 - G_t) \cdot \pi_\theta(\cdot|s_t) + G_t \cdot I_{\bar{a}}(\cdot). \tag{8}$$

Dabei ist $I_{\bar{a}}(\cdot)$ ein One-Hot-kodierter Vektor, der die tatsächlich ausgewählten Aktionen $\bar{a}$ darstellt. Aufgrund der Monte-Carlo-Natur weist REINFORCE eine hohe Varianz bei der Schätzung der Gradienten auf, was die Konvergenz verlangsamt oder instabil machen kann [12]. Daher werden häufig Methoden zur Varianzreduzierung wie Baselines oder eine sinkende Lernrate eingesetzt, um die Leistungsfähigkeit von REINFORCE zu verbessern [13].

5 Experimente

Die Robustheit von RL-Agenten ist in physikalischen Systemen von hoher Relevanz, da diese Systeme häufig von unerwarteten Störungen und Unsicherheiten betroffen sein können [6]. Daher ist es wichtig, Agenten auf ihre Robustheit und Anpassungsfähigkeit gegenüber solchen Störungen zu untersuchen. Ziel ist es zunächst, jeweils einen REINFORCE-Agenten und einen DQN-Agenten ohne Unsicherheiten (Vanilla-Agenten) in der ursprünglichen LunarLander-Umgebung zu trainieren und diese Modelle abzuspeichern. Anschließend wird die Anpassungsfähigkeit dieser gespeicherten Modelle in der modifizierten Umgebung mit Triebwerksausfällen untersucht, indem diese ohne vorheriges Training mit den Umgebungen interagieren. Danach werden die Vanilla-Agenten in der Umgebung mit Unsicherheiten trainiert und getestet.

5.1 LunarLander-Umgebung

In der LunarLander-Umgebung besteht die Aufgabe darin, eine Mondlandefähre schnell und möglichst genau auf einer vordefinierten Landefläche zu landen. Dabei sollte die Fähre während des Fluges keine hohe Schieflage aufweisen und die Nutzung der Triebwerke möglichst gering halten, um energieeffizient zu operieren. Eine erfolgreiche Landung zeichnet sich durch eine präzise Positionierung der Fähre auf der vorgesehenen Landefläche sowie durch eine geringe Neigung und Geschwindigkeit beim Aufsetzen aus.

5.2 Simulierte Triebwerksausfälle

Nachdem die Agenten in der Standard-LunarLander-Umgebung trainiert wurden, werden Unsicherheiten in Form von simulierten Triebwerksausfällen eingeführt. Zur Untersuchung der Robustheit und Anpassungsfähigkeit der Agenten werden zwei Szenarien mit simulierten Triebwerksausfällen definiert:

Szenario 1: Ausfall des Haupttriebwerks
In diesem Szenario wird das zentrale (Haupt-) Triebwerk mit einer Wahrscheinlichkeit von 20 % deaktiviert, sodass selbst bei einer entsprechenden Aktion des

Agenten kein Schub erzeugt wird. Stattdessen erfolgt eine Null-Aktion, und keines der Triebwerke zündet. Dieses Szenario testet, wie der Agent mit Ausfällen eines zentralen Triebwerks umgeht.

Szenario 2: Zufällige Triebwerksausfälle
In diesem Szenario wird die vom Agenten gewählte Aktion mit einer Wahrscheinlichkeit von 80 % ausgeführt, während sie mit einer Wahrscheinlichkeit von 20 % durch eine Null-Aktion ersetzt wird, bei der keines der Triebwerke zündet. Somit wird untersucht, wie robust die Agenten gegenüber allgemeiner systemischer Unsicherheit reagieren.

6 Modelle

6.1 DQN-Modell

Das DQN-Modell basiert auf einem neuronalen Netz mit zwei verdeckten Schichten, die jeweils 128 Neuronen nutzen und die ReLU-Aktivierungsfunktion verwenden. Das Training erfolgt mit einer Batchgröße von 64, wobei der Adam-Optimizer mit einer Lernrate von 0.0005 verwendet wird. Der Discount-Faktor beträgt 0,99, um zukünftige Belohnungen entsprechend zu gewichten. Zur Exploration wird eine epsilon-greedy-Strategie eingesetzt. Zu Beginn startet dieser mit einem Wert von 1.0 und reduziert sich exponentiell über die Episoden hinweg:

$$\epsilon = \max\left(0.01, 1.0 \times 0.996^{\text{episode}}\right) \tag{9}$$

Dies ermöglicht eine schrittweise Reduzierung der Exploration zugunsten einer verstärkten Nutzung der gelernten Strategie. Die gesamte Trainingsdauer umfasst 1000 Episoden, um sicherzustellen, dass der Agent ausreichend Zeit hat, eine sichere Landung in der LunarLander-Umgebung zu erlernen.

6.2 REINFORCE-Modell

Das REINFORCE-Modell wird mit einer Baseline kombiniert. Das neuronale Netz des REINFORCE-Modells besteht aus zwei Schichten mit 64 bzw. 128 Neuronen und nutzt die ReLU-Aktivierungsfunktion. Adam-Optimizer mit einer Lernrate von 0.001 wird verwendet. Um zukünftige Belohnungen angemessen zu berücksichtigen, wird der Discount-Faktor 0,99 verwendet. Im Gegensatz zu DQN wird bei REINFORCE keine epsilon-greedy-Strategie eingesetzt, da das Verfahren direkt eine Policy lernt. Stattdessen wird die Lernrate linear reduziert, um eine stabilere Konvergenz zu ermöglichen, diese wird schrittweise von ihrem ursprünglichen Wert auf 20 % des Startwerts über die gesamte Trainingsdauer von 2500 Episoden reduziert.

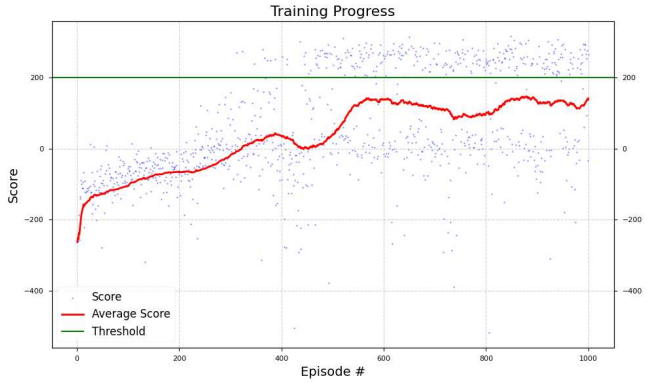


Abbildung 2: Trainingsverlauf DQN in Vanilla-LunarLander-Umgebung

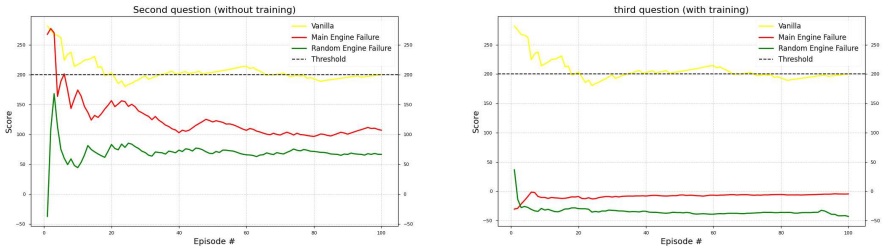
7 Ergebnisse

7.1 DQN

7.1.1 LunarLander-Umgebung

Abbildung 2 zeigt den Trainingsverlauf des DQN-Agenten. Die Episoden-Scores (blaue Punkte) zeigen über den gesamten Trainingsverlauf hinweg eine hohe Variabilität, dennoch lässt sich eine klare Lernentwicklung erkennen. Die durchschnittliche Belohnung (rote Linie) nimmt kontinuierlich zu und deutet darauf hin, dass der Agent mit zunehmender Trainingsdauer eine bessere Steuerstrategie entwickelt. Nach etwa 250 Episoden erzielt der Agent erstmals durchschnittlich positive Scores, wobei bis Episode 600 eine deutliche Leistungssteigerung zu beobachten ist. Im weiteren Verlauf treten zwar Schwankungen auf, die Gesamtperformance stabilisiert sich jedoch auf einem hohen Niveau von etwa 160 Punkten.

7.1.2 Simulierte Triebwerksausfälle



(a) Ohne zusätzlichem Training (b) Mit zusätzlichem Training

Abbildung 3: DQN in Umgebungen mit Unsicherheiten

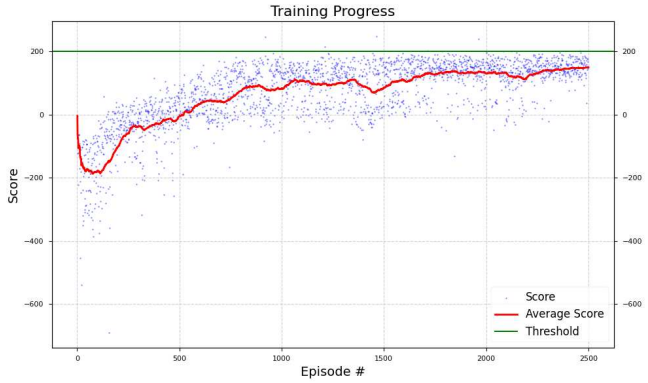


Abbildung 4: Trainingsverlauf REINFORCE in Vanilla-LunarLander-Umgebung

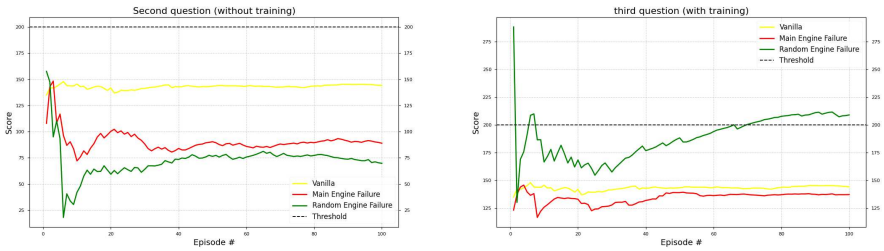
Im ersten Diagramm 3a ist zu erkennen, dass der Agent in der Standardumgebung (gelbe Linie) relativ stabile Rewards mit durchschnittlichen Scores nahe dem Threshold von etwa 200 Punkten aufweist. Wird der Agent jedoch ohne zusätzliches Training mit einem Haupttriebwerksausfall konfrontiert (rote Linie), sinken die durchschnittlichen Scores deutlich von rund 200 auf etwa 100 Punkte. In der Umgebung mit zufälligem Triebwerksausfall (grüne Linie) fällt die Performance weiter ab und stabilisiert sich auf einem niedrigen Niveau bei ca. 60 Punkten. Im zweiten Diagramm 3b zeigt sich, dass der Agent nach erneutem Training in der Umgebung mit Haupttriebwerksausfall (rote Linie) keine signifikante Verbesserung erzielt, sondern eine Verschlechterung, die Scores bleiben nahe 0 Punkten. Gleiches gilt für zufällige Triebwerksausfälle (grüne Linie), bei denen die Performance ebenfalls unter 0 Punkte fällt.

7.2 REINFORCE

7.2.1 LunarLander-Umgebung

Der Trainingsverlauf des REINFORCE-Agenten (Abbildung 4) in der Lunar Lander-Umgebung zeigt eine Lernkurve mit anfänglicher hoher Variabilität in den Scores (blaue Punkte), die sich im weiteren Verlauf zunehmend stabilisiert. Zu Beginn erreicht der Agent negative Scores unter -200 Punkten, ähnlich wie DQN, verbessert sich jedoch rasch in den ersten 500 Episoden auf Werte nahe 0. Dieser frühe Lernfortschritt wird von einem kontinuierlichen, aber schwankenden Anstieg der durchschnittlichen Belohnung begleitet, bis sich nach etwa 1.500 Episoden eine temporäre Performance-Senkung zeigt. Die durchschnittliche Belohnung (rote Linie) in Abbildung 4 zeigt, dass der Agent im weiteren Verlauf ein stabiles Niveau erreicht und sich dem Threshold von 200 Punkten annähert. Die Lernkurve weist nach etwa 2.000 Episoden eine hohe Stabilität auf. Insgesamt bestätigt sich, dass REINFORCE in der Lage ist, die LunarLander-Umgebung erfolgreich zu erlernen, allerdings mit einer höheren Anzahl an benötigten Episoden im Vergleich zu DQN.

7.2.2 Simulierte Triebwerksausfälle



(a) Ohne zusätzlichem Training (b) Mit zusätzlichem Training

Abbildung 5: REINFORCE in Umgebungen mit Unsicherheiten

Im ersten Diagramm 5a bleibt die Vanilla-Performance des Agenten relativ stabil bei etwa 140 Punkten, jedoch unterhalb des Threshold von 200 Punkten. Wenn der Agent ohne zusätzliches Training mit einem Haupttriebwerksausfall konfrontiert wird, sinkt seine Performance auf etwa 90 Punkte. Auch in der Umgebung mit zufälligem Triebwerksausfall fällt die Performance auf ein noch niedrigeres Niveau von unter 75 Punkten. Dieser Einbruch ist im Vergleich zu DQN weniger drastisch, da DQN unter denselben Bedingungen nahezu auf 0 Punkte fällt. Im zweiten Diagramm 5b ist zusehen, dass der REINFORCE-Agent nach zusätzlichem Training in der fehlerhaften Umgebung seine Performance verbessern kann. Insbesondere in der Umgebung mit zufälligem Triebwerksausfall steigt der Score stark an und übertrifft sogar den Threshold von 200 Punkten, was auf eine erfolgreiche Anpassung hinweist. In der Umgebung mit Haupttriebwerksausfall stabilisiert sich die Performance zwar unterhalb des ursprünglichen Vanilla-Werts, bleibt jedoch auf einem Niveau von ca. 130 Punkten.

8 Diskussion

In dieser Arbeit wurde untersucht, wie sich die beiden RL-Ansätze DQN (wertbasiert) und REINFORCE (policybasiert) in unterschiedlichen Unsicherheiten verhalten. Die Ergebnisse zeigen, dass DQN schneller in einer stabilen Umgebung lernen kann, jedoch Schwierigkeiten mit unerwarteten Veränderungen hat, während REINFORCE eine höhere Anpassungsfähigkeit in dynamischen Umgebungen aufweist. Im Folgenden werden die Forschungsfragen diskutiert und die Ergebnisse in den Kontext bestehender Arbeiten eingeordnet.

1. Forschungsfrage: Wie unterscheiden sich die beiden Verfahren hinsichtlich ihrer Fähigkeit, die LunarLander-Umgebung ohne Unsicherheiten effektiv zu lernen?

Die Analyse der Lernkurven zeigt deutliche Unterschiede zwischen den beiden Verfahren. Der DQN-Agent erzielte bereits nach etwa 600 Episoden eine durchschnittliche Belohnung von etwa 180 Punkten, während der REINFORCE-Agent

erst nach 2.000 Episoden ein stabiles Leistungsniveau erreichte. Dies deutet darauf hin, dass DQN die Umgebung schneller erlernt und weniger Episoden benötigt als REINFORCE. Diese Beobachtung deckt sich mit den Ergebnissen von Sadavarte et al. und Shen, die ebenfalls feststellten, dass wertbasierte Methoden in der LunarLander-Umgebung effizienter lernen als policybasierte Ansätze.

Ein weiterer Unterschied zeigt sich in der Stabilität der Lernkurven. Während REINFORCE nach der initialen Lernphase eine relativ konstante Performance beibehält, zeigt DQN eine stärkere Schwankung während des gesamten Trainingsprozesses. Dies widerspricht den Arbeiten von Sadavarte et al. [2] und Shen [4], die DQN als stabiler beschrieben haben. Eine mögliche Ursache für diese Abweichung könnte in der verwendeten linearen Abnahme der Lernrate bei REINFORCE liegen. Diese Anpassung führt dazu, dass sich die Aktualisierung der Gradienten im Verlauf des Trainings abschwächt, wodurch das Modell weniger anfällig für drastische Veränderungen wird und langfristig eine stabilere Performance erzielt. Dies könnte darauf hindeuten, dass DQN empfindlicher auf kurzfristige Variationen in den Q-Werten reagiert, was möglicherweise durch den Einsatz eines Replay-Buffers und die schrittweise Aktualisierung der Q-Funktion verursacht wird. Im Gegensatz dazu führt die schrittweise Reduzierung der Lernrate bei REINFORCE dazu, dass sich das Verhalten des Agenten über längere Zeiträume stabilisiert, da die Aktualisierungen der Strategie mit zunehmender Trainingsdauer immer feiner werden. Dadurch könnte REINFORCE insgesamt robuster gegenüber plötzlichen Veränderungen im Training sein, während DQN eher zu inkonsistenten Schwankungen in der Lernkurve neigt.

Es lässt sich festhalten, dass DQN in einer stabilen Umgebung ohne Unsicherheiten schneller konvergiert und höhere finale Belohnungen erreicht, während REINFORCE einen langsameren, aber stabileren Lernprozess durchläuft. Diese Unterschiede deuten darauf hin, dass DQN für klar definierte, deterministische Aufgaben mit fester Dynamik vorteilhafter sein könnte, während REINFORCE potenziell besser für sich ändernde oder unsichere Umgebungen geeignet ist.

2. Forschungsfrage: Wie robust zeigen sich die Verfahren gegenüber simulierten Triebwerksausfällen?

Zur Beantwortung dieser Fragestellung wurden die zuvor in einer Umgebung ohne Triebwerkausfälle trainierten Agenten in zwei neuen Szenarien getestet: dem Ausfall des Haupttriebwerks (Main Engine Failure) und zufälligen Triebwerksausfällen (Random Engine Failure). Abbildung 3a zeigt die durchschnittliche Belohnung über 100 Episoden für DQN und Abbildung 5a für REINFORCE und verdeutlicht die Auswirkungen der Störungen auf die Agentenleistung. In den Szenarien mit Triebwerksausfällen (rote und grüne Linien) kommt es zu einem deutlichen Leistungseinbruch. Durch die Störungen verlieren beide Verfahren signifikant an Performance, wobei die zufälligen Triebwerksausfälle einen stärkeren negativen Einfluss haben als der Haupttriebwerksausfall. Die beobachteten Unterschiede zwischen den Verfahren lassen sich durch die grundlegenden Eigenschaften wertbasierten und policybasierten Lernens erklären. Während DQN auf der Schätzung von Q-Werten in einer festen Umgebung basiert und

dadurch anfällig für plötzliche Veränderungen in der Dynamik ist [6], optimiert REINFORCE direkt eine Politik und zeigt dadurch eine höhere Flexibilität gegenüber Unsicherheiten [4], auch wenn dies auf Kosten einer insgesamt niedrigeren Performance geschieht.

Stärker betroffen ist DQN, da es feste Q-Werte gelernt hat. Seine Entscheidung beruht darauf, dass bestimmte Aktionen immer die gleiche Belohnung generieren. In einer Umgebung mit zufälliger Nichtfunktionalität von Triebwerken sind die vorher gelernten Q-Werte jedoch nicht mehr zuverlässig, da die gleiche Aktion plötzlich ein anderes Ergebnis haben kann. Das führt zu inkonsistenten Steuerentscheidungen und macht es dem Agenten schwer, ein stabiles Verhalten zu haben. Die beobachteten Ergebnisse entsprechen dennoch den Erkenntnissen von Gadgil et al. [7], die gezeigt haben, dass ein Vanilla-DQN-Agent auch ohne zusätzliches Training mit zufälligen Triebwerksausfällen noch positive durchschnittliche Belohnungen erzielen kann.

REINFORCE zeigt ebenfalls eine Leistungsminderung, jedoch im geringeren Ausmaß als DQN. Da REINFORCE eine Policy direkt optimiert, kann es mit etwas mehr Flexibilität auf Unsicherheiten reagieren. Allerdings wurde die ursprüngliche Policy in einer stabilen Umgebung gelernt, sodass auch hier plötzliche und unerwartete Fehlfunktionen zu einer Performance-Reduktion führen.

Insgesamt zeigt sich, dass REINFORCE weniger drastische Einbrüche als DQN zeigt. Die Ergebnisse legen nahe, dass DQN in stabilen, deterministischen Umgebungen überlegen ist, jedoch erhebliche Schwierigkeiten bei der Anpassung an unerwartete Störungen zeigt. REINFORCE hingegen bleibt stabiler unter Unsicherheiten, was darauf hindeutet, dass policybasierte Verfahren eine bessere Wahl für adaptive Steuerungsprobleme in dynamischen Umgebungen sein könnten.

3. Forschungsfrage: Welches Verfahren weist bei simulierten Triebwerksausfällen eine schnellere Anpassungsfähigkeit auf?

Um die Anpassungsfähigkeit von DQN und REINFORCE an veränderte Umweltbedingungen zu untersuchen, wurden die zuvor trainierten Agenten gezielt in gestörten Szenarien trainiert und anschließend getestet. Die vorherigen Ergebnisse zeigen bereits Unterschiede in der Stabilität der beiden Verfahren bei unerwarteten Störungen. Nun liegt der Fokus auf der Frage, ob DQN und REINFORCE ihre Steuerstrategien effektiv modifizieren können, um sich an den Ausfall von Triebwerken anzupassen. Abbildung 3b zeigt die Ergebnisse für DQN, während Abbildung 5b die entsprechenden Ergebnisse für REINFORCE zeigt.

Die Ergebnisse zeigen, dass sich die bereits zuvor beobachteten Unterschiede zwischen den beiden Verfahren durch das zusätzliche Training weiter verstärken. DQN bleibt auch nach dem Training in der neuen Umgebung weiterhin eingeschränkt. Insbesondere bei zufälligen Triebwerksausfällen ist zu erkennen, dass die Performance nahezu unverändert auf einem niedrigen oder sogar negativen Niveau verharrt. Der Haupttriebwerksausfall erweist sich für DQN als etwas besser kompensierbar, jedoch bleibt die Leistung auch hier weit unter der ur-

sprünglichen Vanilla-Performance. Dies legt nahe, dass DQN mit zusätzlichem Training keine robuste Steuerstrategie für dynamisch veränderte Umgebungen entwickeln kann.

Im Gegensatz dazu zeigt REINFORCE eine deutlich höhere Lernfähigkeit. Bereits in der Beantwortung der zweiten Forschungsfrage wurde festgestellt, dass REINFORCE flexibler auf Umweltveränderungen reagiert. Durch das zusätzliche Training wird die Performance beim Haupttriebwerksausfall stabilisiert und eine solide Steuerstrategie entwickelt. REINFORCE zeigt bei zufälligen Triebwerksausfällen eine deutliche Verbesserung. Nach einer anfänglichen Anpassungsphase steigen die Scores über das ursprüngliche Vanilla-Niveau, was darauf hindeutet, dass REINFORCE nicht nur die Unsicherheiten der neuen Umgebung ausgleichen kann, sondern diese sogar nutzt, um eine effektivere Steuerstrategie als zuvor zu entwickeln.

Zusammenfassend zeigt sich, dass REINFORCE im direkten Vergleich eine signifikant höhere Anpassungsfähigkeit an simulierte Triebwerksausfälle aufweist als DQN. Während DQN nach einem Training kaum Fortschritte erzielt und bei zufälligen Störungen weiterhin keine stabilen Verbesserungen zeigt, verbessert sich REINFORCE kontinuierlich und kann sich sogar über das ursprüngliche Leistungsniveau hinaus verbessern.

9 Fazit & Ausblick

Insgesamt lässt sich feststellen, dass DQN in stabilen Umgebungen besser abschneidet und höhere Belohnungen erzielt als REINFORCE. Bei unerwarteten Störungen erweist sich jedoch REINFORCE als anpassungsfähiger, insbesondere nach zusätzlichem Training. Ohne erneutes Training erleiden beide Methoden Leistungseinbußen, wobei DQN stärker betroffen ist. Nach erneutem Training kann REINFORCE seine Performance deutlich verbessern und sogar das ursprüngliche Niveau übertreffen, während DQN sich kaum an die veränderten Bedingungen anpassen kann. Dies legt nahe, dass policybasierte Verfahren besser für dynamische und unsichere Umgebungen geeignet sind, während wertbasierte Methoden in deterministischen Szenarien ihre Stärken ausspielen.

Zukünftige Arbeiten könnten die Robustheit dieser Verfahren weiter in Bezug auf Triebwerkausfälle untersuchen, indem sie unkontrolliertes Zünden oder unterschiedlich starke Schubstöße simulieren. Zudem wäre die Entwicklung hybrider Ansätze interessant, die die Effizienz wertbasierter Methoden mit der Anpassungsfähigkeit policybasierter Verfahren kombinieren. Schließlich könnten fortschrittliche RL-Algorithmen wie Actor-Critic oder PPO dahingehend untersucht werden, ob sie eine bessere Robustheit gegenüber unsicheren Umgebungen aufweisen.

Literatur

[1] Larasmoyo Nugroho, Novanna Rahma Zani, Nurul Qomariyah, Rini Akmeliawati, Rika Andiarti, and Sastra Kusuma Wijaya. Powered Landing Guidance Algorithms Using Reinforcement Learning Methods for Lunar Lander Case. June 2021.

[2] Rohit Sachin Sadavarte, Rishab Raj, and B Sathish Babu. Solving the Lunar Lander Problem using Reinforcement Learning. In *2021 IEEE International Conference on Computation System and Information Technology for Sustainable Solutions (CSITSS)*, pages 1–6, Bangalore, India, December 2021. IEEE.

[3] Håkon Guttulsrud, Mathias Sandnes, and Raju Shrestha. Solving the Lunar Lander Problem with Multiple Uncertainties using a Deep Q-Learning based Short-Term Memory Agent. In *2023 12th International Conference on Computing and Pattern Recognition*, pages 27–33, Qingdao China, October 2023. ACM.

[4] Dingli Shen. Comparison of Three Deep Reinforcement Learning Algorithms for Solving the Lunar Lander Problem. In *2023 International Conference on Data Science, Advanced Algorithm and Intelligent Computing (DAI 2023)*, pages 187–199. Atlantis Press, 2024.

[5] Sindre Benjamin Remman, Bjørn Andreas Kristiansen, and Anastasios M. Lekkas. Deep Reinforcement Learning Behavioral Mode Switching Using Optimal Control Based on a Latent Space Objective, June 2024.

[6] Salman Shah and Nianmin Yao. Deep Reinforcement Learning for Unpredictability-Induced Rewards to Handle Spacecraft Landing. In *2023 13th International Conference on Information Science and Technology (ICIST)*, pages 73–79, Cairo, Egypt, December 2023. IEEE.

[7] Soham Gadgil, Yunfeng Xin, and Chengzhe Xu. Solving The Lunar Lander Problem under Uncertainty using Reinforcement Learning. 2020.

[8] Farama Foundation. https://farama.org/about, 2025.

[9] Lunar Lander Gymnasium Documentation. https://gymnasium.farama.org/environments-/box2d/lunar_lander.html, 2024.

[10] Volodymyr Mnih, Koray Kavukcuoglu, David Silver, Alex Graves, Ioannis Antonoglou, Daan Wierstra, and Martin Riedmiller. Playing Atari with Deep Reinforcement Learning, December 2013.

[11] Volodymyr Mnih, Koray Kavukcuoglu, David Silver, Andrei A. Rusu, Joel Veness, Marc G. Bellemare, Alex Graves, Martin Riedmiller, Andreas K. Fidjeland, Georg Ostrovski, Stig Petersen, Charles Beattie, Amir Sadik, Ioannis Antonoglou, Helen King, Dharshan Kumaran, Daan Wierstra, Shane Legg, and Demis Hassabis. Human-level control through deep reinforcement learning. *Nature*, 518(7540):529–533, February 2015.

[12] Richard S. Sutton and Andrew G. Barto. *Reinforcement Learning: An Introduction*. Adaptive Computation and Machine Learning Series. The MIT Press, Cambridge, Massachusetts, second edition edition, 2018.

[13] Stephan Pareigis. Algorithmic Foundations of Reinforcement Learning. In Herwig Unger and Marcel Schaible, editors, *Advances in Real-Time and Autonomous Systems*, volume 1009, pages 1–27. Springer Nature Switzerland, Cham, 2024.